A Major Project Report
(Phase-I)

On

AI – Based Network Traffic Analysis

Submitted in partial fulfillment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IILM UNIVERSITY

Session 2026-27

in

Computer Science and Engineering

By

Deepshikha Rani (2410030760)

Himansh Papnai (2410030812)

Bhupesh Singh (2410030786)

Ananya Vashishtha (2410030128)

Under the guidance of Ms. Apoorva Jain

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
IILM UNIVERSITY, GREATER NOIDA

May, 2026

SCHOOL OF COMPUTER SCIENCE AND ENGG.

IILM UNIVERSITY, GREATER NOIDA

CANDIDATE'S DECLARATION

We hereby certify that the work presented in the **Phase-I report** of the project entitled "**AI – Based Network Traffic Analysis**", in partial fulfillment of the requirements for the degree of **Bachelor of Technology** in the **School of Computing Science and Engineering, IILM University**, Greater Noida, is an original work carried out by us under the supervision of **Ms. Apoorva Jain**, School of Computing Science and Engineering, IILM University, Greater Noida.

The work presented in this Phase-I report has not been submitted by us for the award of any other degree or qualification at this or any other institution.

Deepshikha Rani (2410030760)
Himansh Papnai (2410030812)
Bhupesh Singh (2410030786)
Ananya Vashishtha (2410030128)

This is to certify that the above statement made by the candidates is correct to the best of my knowledge.

Signature of Guide Name and Details of Guide

Name: Apoorva Jain

Designation: Assistant Professor

School: School of Computing Science and Engineering

IILM University, Greater Noida

Table of Contents

SL. No	Section	Page no.
	Abstract	1
1	Problem Definition	2
2	Literature Survey	4
3	Objectives	8
4	Scope	9
5	Feasibility Study	11
	References	14
	Plagiarism Report	16

ABSTRACT

Today's networks are much more than just for web browsing. The number of cloud services, mobile devices, remote applications and background processes running simultaneously renders manual analysis of network logs increasingly difficult. Signature-based security tools continue to be beneficial for known attacks, but rely on predefined rules and can leave administrators with a ton of data to examine. In this project, a real-world application of machine learning to solve this problem is examined by analysing data from the network flow, and determining whether there is any traffic in the network that can be considered for intervention. The work is only limited to a scope that will be attainable and demonstrable as a B.Tech major project.

The main reason for using CICIDS2017 is that is has labeled benign and malicious traffic and flow-level features, which are pertinent for the machine-learning experiments [1]. Data will be cleaned prior to training and a small number of useful features retained so that same processing can be applied during monitoring. Not only will overall accuracy be used to compare a small set of supervised models (Decision Tree, Random Forest and Logistic Regression) but also precision, recall, F1-score, confusion matrix and prediction time will be used. Previous work has demonstrated that tree-based techniques offer a good compromise between detection performance and the cost of computation in flow-based intrusion detection [4, 6]. The best performing practical model will then be linked to captured or replayed network-flow data. It will be presented in a basic dashboard with traffic summaries, alerts and event logs. The aim of the project is thus not to suggest a new learning algorithm, but to create a clear and reproducible prototype that links data preparation, model evaluation and near-real-time monitoring of networks into a single system that works.

Keywords: network traffic monitoring, machine learning, intrusion detection, flow features, CICIDS2017, Random Forest.

1. PROBLEM DEFINITION

1.1 Background

A computer network is used for a myriad of activities: applications, background services, user requests, and sometimes even malicious connections. Hand reviewing packets and logs becomes cumbersome and time-consuming as traffic volume increases. Signature-based intrusion detection is able to identify threats with known patterns, but relies on the rules, which need to be updated as attacks evolve. Machine learning can assist in this by learning patterns from labelled network-flow data and applying the patterns to detect the normal and non-normal traffic.

The primary difficulty with this project is to take a machine-learning experiment that is not online, and make it useful for monitoring. If the data isn't consistently prepared for the classifier, there are too many features, the predictions are too slow or too many alerts are generated if the traffic is normal, then the classifier can have a high accuracy on the dataset, but a low accuracy in practice. CICIDS2017 results indicate that flow-based classifiers can be effective with tree-based classifiers, and reducing unnecessary features can reduce the processing overhead [4]. There are also works that have demonstrated the effectiveness of the conventional machine-learning models when carefully prepared data [5, 6]. This makes a light and end-to-end design more appropriate to this present project than a very complicated model that is hard to integrate and explain.

1.2 Problem Statement

This project deals with the following problem:

- Manually going through a lot of network traffic and logs is challenging, particularly if fast decisions need to be made.
- The only major downside of signature based is that they are largely rule based, that is to say, anything not matching any of those patterns will need further analysis.
- However, good models on the stored benchmark data do not necessarily perform well for monitoring, other factors, e.g., preprocessing, feature selection, prediction speed or false-positive behaviour are also important.
- A small and efficient system should be designed to distinguish between regular and suspicious traffic flows and provide simple alerts and logs of traffic classification,

with no need to be complicated.

1.3 Proposed Approach

The primary source of labelled data for CICIDS2017 will be benign traffic, several attack categories and flow-based measurements that can be directly handled in python [1]. The first application of the system will be in the binary classification class with the traffic classified as legitimate or malicious. This makes it easier to validate and more appropriate for a near real time prototype. If it is a fairly stable main binary pipeline and the work is not too much, a limited multi class experiment could be attempted in the future.

The shortlisted models will undergo the same cleaning and feature preparation as before, and then be prepped, trained and tested in the same manner. Precision, recall, F1-score, results for the confusion matrix, behaviour and time of false positive will be used as the basis for the final choice. In monitoring mode, the captured or replayed traffic will be converted into the same format as the training traffic, and then be fed to the saved model. Basic traffic information will be shown on the dashboard, as well as the expected result, alert status and a list of the traffic detected with a time stamp. This is a tight focus because it is based on a complete and usable pipeline and not a new learning algorithm.

1.4 Key Challenges Addressed

- There could be missing data, infinite data, duplicate data, and extremely imbalanced class sizes in public traffic data. Prior to training, these issues should be addressed to make sure that the model is not being trained with incorrect information.
- Using all features can make training and prediction slower, but can offer no extra information. Only features with meaningful correlation or model based importance will be kept by simple correlation or model based importance.
- When capturing and replaying traffic to classify it, the feature names, units and preprocessing operations on which the traffic is trained need to be duplicated.
- Final presentation must be capable of being read. Predictions will then be presented in the form of 'brief alerts' and 'event records', as opposed to model output.

2. LITERATURE SURVEY

2.1 Network Traffic Data and Benchmark Datasets

The reliability of an intrusion-detection model is strongly related to that of the data on which it is trained. The IDS datasets were evaluated according to some criteria presented by Gharib et al. [2] which include the diversity of normal traffic, extent of attack coverage, fullness of captured volume and quality of labels. This is crucial as a model can't learn behaviour that is not present or poorly represented in the training data. Later, Sharafaldin et al. added another set of activities called CICIDS2017 which also includes some benign activities and multiple attack scenarios collected over a number of days [1]. It also can accept labelled flow records created with CICFlowMeter as well as packet capture. Information like flow duration, numbers of packets, transferred bytes, timing information is already tabulated. This makes the CICIDS2017 a perfect fit for the current project: Experiments can begin with the structured CSV files, but can still be based on measurements directly related to real network flows.

Another well-known dataset used, introduced by Moustafa and Slay, is UNSW-NB15 which consists of normal and attack traffic [3]. It will be beneficial to know if outcomes obtained with CICIDS2017 apply to another set of data. However, if several large data sets were produced in the first phase, there would be lots of pre-processing and testing. CICIDS2017 will thus continue to be the standard, and a second set of data will be considered optional validation and considered as part of the implementation.

2.2 Flow-Based Features and Encrypted Traffic

The content of a message does not necessarily need to be inspected to analyse network traffic. Habibi Lashkari et al. showed how to take advantage of the timing and flow characteristics of encrypted Tor traffic to distinguish them [16]. This is helpful to the system that is being proposed since a payload could be encrypted or just be something that's not appropriate to look at. The prototype will thus incorporate flow metadata like duration, packet direction, packet-size statistics, inter-arrival behaviour and connection-level measurements. This also incorporates what was provided in CICIDS2017 and the emphasis is on traffic behaviour and not private material.

2.3 Comparison of Machine-Learning Approaches

The reviewed studies do not indicate that a more complex model is necessarily better than a simpler model for an intrusion detector. Rodriguez et al. investigated various classifiers in

the context of flow based detection and found that the tree classifiers were good contenders. They also showed that reducing the number of features was correlated with reduction in testing times and by adding flow information from Zeek they were able to show that it is possible to process flows at a fast rate [4]. This will be directly applicable to the current project because the chosen model will need to be both in an offline experiment and in the monitoring pipeline.

Other studies also impart a like practical message. Roy et al. developed a lightweight supervised IDS by preprocessing, feature reduction and sampling, that has taken training cost and false alerts into account [5]. The researchers Abbas et al. [6] compared the conventional machine-learning techniques and ensemble techniques on CICIDS2017 and demonstrated that useful binary and multi-class detection can be possible without depending totally on deep neural networks. The results validate the appropriateness of the base models used widely – Decision Tree, Random Forest and Logistic Regression – and only K-Nearest Neighbours was retained when it was necessary for a simple comparison.

Another issue that is mentioned in the literature often is feature selection as network datasets may have a lot of correlated and low value attributes. In the past few years, Disha and Waheed have been researching on the Random-Forest based feature selection to detect intrusion [7] and Prasad et al. have stated that the reduction of feature space reduces training and computation effort [8]. Similar feature-importance approaches were applied by Alduailij et al., in their experiments with DDoS attacks, and they achieved high performance results using fewer inputs with Random Forest [10]. The one thing that comes from this project is that when there is a simple method that is useful and is correlated enough to create a compact feature set, then it is sufficiently enough and not necessary to introduce the optimisation heavy stage.

2.4 Class Imbalance and Evaluation Measures

Class imbalance is another problem that can skew the results of intrusion detection. A high overall accuracy can be deceiving, as some attacks may not occur very frequently compared to benign traffic and common attacks. In experiments with the CICIDS2017 and CICIDS2018, Lalduhsaka et al. reported not only classification performance but also false-alarm behaviour [9] which backs up the notion that one need not restrict oneself to one single headline score. Gupta et al. was a more complicated cost-sensitive deep-learning solution [14] for imbalance. It doesn't have to be that complicated but it is a concern here. The project will explore class frequencies, consider stratified train-test splits, and consider class

weighting or controlled re-sampling only when required. In addition to the accuracy, precision, recall and F1 score will be reported, and confusion matrices will be provided.

2.5 Real-Time and Near-Real-Time Analysis

Although it works well with some stored data, it does not guarantee to work well with real-time monitoring data. Chowdhury et al. tested an ID pipeline on an actual testbed that they trained using CICIDS2017 [11] on a real-time testbed. Their full approach is more complicated than this project can be, but the concept of capturing the traffic, converting it into the proper feature format, and classifying it with a reasonable delay is crucial. The whole idea of an end-to-end is the same with the proposed prototype, just on a smaller scale. Traffic can be captured in a controlled local environment and replayable packet or flow data can be replayed for repeatable tests to be more useful.

Claims of performance need to also be realistic when the training environment changes. Leevy et al. demonstrated that the results may differ between different feature sets and learners [12]. Vinayakumar et al. and Yin et al. have shown that deep and recurrent neural methods are able to provide good detection of intrusions [13, 15] but their design is more complex and requires more tuning and computation resources than the core prototype. There is also recent work on explainable IDS that highlights the importance of providing some explanation to the user about what the security prediction means [17, 18]. In this project, any explanation will not be complicated like the case of a separate complex XAI pipeline, but rather it will be light: When the chosen model is suitable, it will show influential features, when it is not, it will not.

2.6 Comparative Summary of Selected Studies

Study	Method / Data	Main observation	Relevance to this project
[1] Sharafaldin et al., 2018	CICIDS2017; labelled flow records	Mixed benign and attack traffic in a structured benchmark.	Main dataset and flow format.
[4] Rodriguez et al., 2022	ML classifiers; CICIDS2017; Zeek	Tree models were strong; fewer features reduced test time.	Supports RF/DT and fast flow processing.
[5] Roy et al., 2022	Lightweight supervised IDS; reduced features	Lower-cost processing remained effective without a heavy model.	Supports a lightweight design.
[6] Abbas et al., 2022	Standard ML and ensemble; CICIDS2017	Conventional models were useful for binary and multi-class detection.	Provides baseline algorithms.
[7] Disha and Waheed, 2022	Random-Forest feature selection	Selecting informative attributes improved the modelling pipeline.	Supports importance-based reduction.
[9] Lalduhsaka et al., 2022	Ensemble; CICIDS2017/CICIDS2018	Evaluation included false-alarm behaviour, not accuracy alone.	Supports multiple evaluation metrics.

Study	Method / Data	Main observation	Relevance to this project
[10] Alduailij et al., 2022	DDoS detection; feature importance; MI	Random Forest remained strong after feature reduction	Relevant to feature and model choice.
[11] Chowdhury et al., 2023	CICIDS2017 plus real-time testbed	Connected capture, feature preparation and classification in one workflow.	Supports live/replay integration.
[12] Leevy et al., 2021	Multiple learners and feature sets	Results varied with preparation, features and data environment.	Encourages cautious generalisation.
[13] Vinayakumar et al., 2019	Deep-learning IDS; multiple datasets	Deep models are capable but require a more demanding setup.	Comparison only; not core implementation.

2.7 Research Gap and Direction Adopted

The literature overall indicates that it is not really interesting to train another model with the goal of achieving the maximum CICIDS2017 accuracy if it is done alone. Rather, a more useful goal for this large effort is to tie together the parts that are typically examined individually: reproducible preprocessing, a small feature set, a well-designed model comparison, and a small monitoring pipeline that can be used in real or replay mode. The system will hence be evaluated with respect to the possibility of applying the same preprocessing during training and monitoring, the management of false alarms and the clarity of the information conveyed by the dashboard.

There are also studies investigating real-time testing and generalisation which attest to the avoidance of equating a high benchmark score with production performance [11], [12]. Future work is left for more advanced areas like SDN orchestration, adaptive sampling, continuous retraining and large deep-learning ensembles as these features would require major development investments and were not required to prove the primary network-analysis workflow.

3. OBJECTIVES

The project has been kept to three practical objectives that can be completed and demonstrated within the major-project timeline:

1. Prepare the CICIDS2017 flow data by cleaning the records and selecting a small set of useful features that can be reproduced during monitoring.
2. Train and compare a few lightweight supervised machine-learning models, and select the most suitable one using precision, recall, F1-score, confusion-matrix results and prediction time.
3. Integrate the selected model with captured or replayed network-flow data and present benign/suspicious results, alerts and event logs through a simple monitoring dashboard.

These objectives keep the project focused on a complete working prototype rather than on an unnecessarily large list of features. The expected contribution is a reproducible path from labelled flow data to a trained classifier and then to a simple monitoring interface. The project will therefore be evaluated mainly on reliability, reasonable processing time and clarity of the results.

4. SCOPE

4.1 In-Scope Work

- The primary labelled dataset for training, testing and comparison of models will be CICIDS2017. The system will not examine any application contents but will examine the flow statistics and connection metadata.
- Data pipeline to include cleaning, preparation of labels, checks for class-distribution, scaling if necessary and clear training/testing split.
- A limited number of supervised classifiers from the standard Python libraries will be used to compare.
- Two approaches will be used for feature reduction: simple methods like correlation checks and Random Forest feature importance.
- The primary activity will be benign versus suspicious classification. If the core binary system is stable, multi-class testing will be tried.
- Near real time testing will be conducted using traffic captured in a controlled local environment or replayed traffic flow, ensuring that any experiments conducted are safe and repeatable.
- Traffic counts, classification results, alerts and event history will be displayed in a simple dashboard.
- Accuracies, precisions, recall and f1-scor will be evaluated, the confusion matrix will be used to check the performance, and false positive behaviour and basic timing measurements will be used.

4.2 Out-of-Scope Work

- The prototype will not actively deny any hosts, modify the firewalls or automatically reply to any traffic detected, but will just analyse and report.
- Private message content is not part of the project and so the construction of the protected payload is not encrypted. Only flow level information will be utilized.
- There is no requirement to implement large neural networks, generative-AI models or train on GPUs.
- The following are not planned for Phase-I: SDN controller clustering, adaptive routing, dynamic sampling and distributed orchestration.
- The first version will not have the ability to learn or automatically retrain in the online mode. These could be talked about as possible additions in the future.
- The project will not claim guaranteed detection of unseen or zero-day attacks. Any anomaly-detection extension will remain optional.

- Testing will not be performed on a network of a production organisation, but on public data or an authorised local environment.

4.3 Expected Deliverables

- Documented and repeatable pre-processing pipeline for the selected CICIDS2017 data. A shortlist of the models and the final feature set.
- A stored classification model, and consistent preprocessing that can be reused.
- A local capture/replay module, which translates the flow information into model input and can return to a prediction.
- For simplicity, a user-friendly traffic summary, alert and event history view.
- A final evaluation of performance, assumptions, limitations and potential future enhancements.

5. FEASIBILITY STUDY

5.1 Technical Feasibility

The proposed system is technically viable and the use of software and hardware is readily available to students. There are well-developed libraries in Python, such as pandas, NumPy, and scikit-learn, that are perfect for handling data and machine learning tasks.Python offers libraries like pandas, NumPy, and scikit-learn, which are well-developed and ideal for data handling and machine learning. Traffic capture/replay: Wireshark/tshark, PyShark, Streamlit/compatible flow extractor can be used as a monitoring interface: Flask, Streamlit. The core project does not need any special networking hardware, a cluster of GPUs, or an SDN laboratory. In case the entire CICIDS2017 data set is not available in the memory, developments can be started using a subset of the data which is documented and class aware without changing the overall methodology.

A key technical challenge is maintaining consistency of the monitoring data with the training data. To cope with this, the final feature list and preprocessing rules will be settled prior to integration, and the same preprocessing routine will be employed in test and inference. Even if live capture is done, a replay mode will be maintained as replayed traffic provides repeatable conditions for debugging and evaluation.

5.2 Operational Feasibility

In terms of use, the system is easy to operate due to its user friendly design. The student, evaluator or lab administrator should be able to check the network activity and review alerts generated by the model without having to understand the internal machine learning calculations. Basic traffic counts, classification result, detection time and event history will be displayed on the dashboard. If appropriate and feasible with the selected model, significant contributing features can also be displayed. The system only alerts to suspicious activity, rather than blocking traffic, so that if it is wrongly predicted, it is unlikely to impact the legitimate use of the network during testing.

5.3 Economic Feasibility

The project does not pose any undue financial burden. All the main programming tools, the machine-learning libraries, the traffic-analysis tools and the web frameworks are open source, and the benchmark datasets selected are available for academic purposes. Student Laptop or Lab computers can be used for training and development. The lightweight models

are planned and cloud resources are optional. The primary expense is thus development and testing, not paid software, or a dedicated infrastructure.

5.4 Schedule Feasibility

It can be done in phases, making it easier to handle the schedule. The first will take care to prepare the dataset and will finalise the feature pipeline. The second will be responsible for training and comparing the baseline models. The third will link the chosen model to captured or replayed network flows, while the fourth will include the dashboard and event logging. The last step will be focused on controlled testing and documentation. The reason why this order is convenient is that the offline classifier is finished prior to the start of monitoring integration. Even if the continuous capture is longer than expected, the entire process can still be shown through replay-based classification.

5.5 Data, Privacy and Safety Feasibility

Requirements for data and safety are also readily achievable. The training will primarily be based on a public benchmark and not traffic from unrelated users. Local captures for demonstration purposes will only occur in an authorised and controlled environment, and will only demonstrate flow metadata and not the content of any private messages. IP addresses do not need to be kept as permanent training characteristics. A restricted laboratory or isolated configuration will be used to restrict suspicious test traffic when it is generated. The project must be a defensive monitoring project and not an attacking or exploiting project of third-party networks.

5.6 Feasibility Risks and Mitigation

Risk	Mitigation
Dataset size and memory use	Start with selected files or stratified subsets if needed, while keeping the preprocessing script reproducible.
Uneven class distribution	Check class counts first; use class weights or controlled resampling only when it is actually required.
Overfitting to CICIDS2017	Keep a separate hold-out test set and, if time permits, test some separately captured benign traffic.
Excessive false alerts	Tune the selected model conservatively and review precision, recall, the confusion matrix and false-positive counts.
Mismatch in live features	Fix the feature schema early and use the same preprocessing function for training and monitoring.
Delay in real-time integration	Keep replay-based flow analysis as a fallback so the complete system can still be demonstrated reliably.

Overall, the project is viable for a B.Tech major project schedule since it is based on public data and standard Python tools, with a limited number of machine-learning models. The simplicity of the first version, centred on flow based classification and a simple dashboard should enable the group to finish, test and show the full system without having to rely on very expensive infrastructure or very specialised hardware.

REFERENCES

[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP), 2018, pp. 108–116, doi: 10.5220/0006639801080116.

[2] A. Gharib, I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "An Evaluation Framework for Intrusion Detection Dataset," in 2016 Int. Conf. Information Science and Security (ICISS), 2016, pp. 1–6, doi: 10.1109/ICISSEC.2016.7885840.

[3] N. Moustafa and J. Slay, "UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems," in 2015 Military Communications and Information Systems Conference (MilCIS), 2015, pp. 1–6, doi: 10.1109/MilCIS.2015.7348942.

[4] M. Rodriguez, Á. Alesanco, L. Mehavilla, and J. García, "Evaluation of Machine Learning Techniques for Traffic Flow-Based Intrusion Detection," Sensors, vol. 22, no. 23, Art. no. 9326, 2022, doi: 10.3390/s22239326.

[5] S. Roy, J. Li, B.-J. Choi, and Y. Bai, "A Lightweight Supervised Intrusion Detection Mechanism for IoT Networks," Future Generation Computer Systems, vol. 127, pp. 276–285, 2022, doi: 10.1016/j.future.2021.09.027.

[6] A. Abbas, M. A. Khan, S. Latif, M. Ajaz, A. A. Shah, and J. Ahmad, "A New Ensemble-Based Intrusion Detection System for Internet of Things," Arabian Journal for Science and Engineering, vol. 47, pp. 1805–1819, 2022, doi: 10.1007/s13369-021-06086-5.

[7] R. A. Disha and S. Waheed, "Performance Analysis of Machine Learning Models for Intrusion Detection System Using Gini Impurity-Based Weighted Random Forest Feature Selection Technique," Cybersecurity, vol. 5, Art. no. 1, 2022, doi: 10.1186/s42400-021-00103-8.

[8] M. Prasad, S. Tripathi, and K. Dahal, "An Efficient Feature Selection Based Bayesian and Rough Set Approach for Intrusion Detection," Applied Soft Computing, vol. 87, Art. no. 105980, 2020, doi: 10.1016/j.asoc.2019.105980.

[9] R. Lalduhsaka, N. Bora, and A. K. Khan, "Anomaly-Based Intrusion Detection Using Machine Learning: An Ensemble Approach," International Journal of Information Security and Privacy, vol. 16, no. 1, pp. 1–15, 2022, doi: 10.4018/IJISP.311466.

[10] M. Alduailij, Q. W. Khan, M. Tahir, M. Sardaraz, M. Alduailij, and F. Malik, "Machine-Learning-Based DDoS Attack Detection Using Mutual Information and Random Forest Feature Importance Method," Symmetry, vol. 14, no. 6, Art. no. 1095, 2022, doi: 10.3390/sym14061095.

[11] R. Chowdhury, S. Sen, A. Goswami, S. Purkait, and B. Saha, "An Implementation of Bi-Phase Network Intrusion Detection System by Using Real-Time Traffic Analysis," Expert Systems with Applications, vol. 224, Art. no. 119831, 2023, doi: 10.1016/j.eswa.2023.119831.

[12] J. L. Leevy, J. Hancock, R. Zuech, and T. M. Khoshgoftaar, "Detecting Cybersecurity Attacks Across Different Network Features and Learners," Journal of Big Data, vol. 8, Art. no. 38, 2021, doi: 10.1186/s40537-021-00426-w.

[13] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep Learning Approach for Intelligent Intrusion Detection System," IEEE Access, vol. 7, pp. 41525–41550, 2019, doi: 10.1109/ACCESS.2019.2895334.

[14] N. Gupta, V. Jindal, and P. Bedi, "CSE-IDS: Using Cost-Sensitive Deep Learning and Ensemble Algorithms to Handle Class Imbalance in Network-Based Intrusion Detection Systems," Computers & Security, vol. 112, Art. no. 102499, 2022, doi: 10.1016/j.cose.2021.102499.

[15] C. Yin, Y. Zhu, J. Fei, and X. He, "A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks," IEEE Access, vol. 5, pp. 21954–21961, 2017, doi: 10.1109/ACCESS.2017.2762418.

[16] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor Traffic Using Time Based Features," in Proc. 3rd Int. Conf. Information Systems Security and Privacy (ICISSP), 2017, pp. 253–262, doi: 10.5220/0006105602530262.

[17] M. M. Mahmoud, Y. O. Youssef, and A. A. Abdel-Hamid, "XI2S-IDS: An Explainable Intelligent 2-Stage Intrusion Detection System," Future Internet, vol. 17, no. 1, Art. no. 25, 2025, doi: 10.3390/fi17010025.

[18] I. C. Obagbuwa, M. N. Ngafeeson, O. F. Obagbuwa, and A. Tsetse, "Machine Learning and Explainable Artificial Intelligence for Network Intrusion Detection," International Journal of Information Security and Privacy, vol. 20, no. 1, pp. 1–22, 2026, doi: 10.4018/IJISP.402900.

PLAGIARISM REPORT

Vijendra Singh

AI_Based_Network_Traffic_Analysis_Phase_I_Report2

Quick Submit

Quick Submit

IILM University, Greater Noida

Document Details

Submission ID
trn:oid:::1:3634266551

Submission Date
Aug 27, 2026, 10:12 AM GMT+5:30

Download Date
Aug 27, 2026, 10:28 AM GMT+5:30

File Name
AI_Based_Network_Traffic_Analysis_Phase_I_Report2.pdf

File Size
296.1 KB

18 Pages

4,850 Words

26,983 Characters

PLAGIARISM REPORT

5% Overall Similarity

The combined total of all matches, including overlapping sources, for each database

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text

Match Groups

- 16 Not Cited or Quoted 5%
  Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

4% Internet sources
3% Publications
3% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

AI DETECTION REPORT

Vijendra Singh

AI_Based_Network_Traffic_Analysis_Phase_I_Report2

Quick Submit

Quick Submit

IILM University, Greater Noida

Document Details

Submission ID
trn:oid:::1:3634266551

Submission Date
Aug 27, 2026, 10:12 AM GMT+5:30

Download Date
Aug 27, 2026, 10:28 AM GMT+5:30

File Name
AI_Based_Network_Traffic_Analysis_Phase_I_Report2.pdf

File Size
296.1 KB

18 Pages

4,850 Words

26,983 Characters

AI DETECTION REPORT

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is entirely human-generated as AI-generated and likely AI-generated as human-generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.